MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

Cryptology – Lab 4

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cyber Security

5/09/2026

MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Level 1:

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named cryptology.txt. Store each statement on a separate line.

sample statements:
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.
Cryptanalysis is fun :)
I love Cryptanalysis ;)

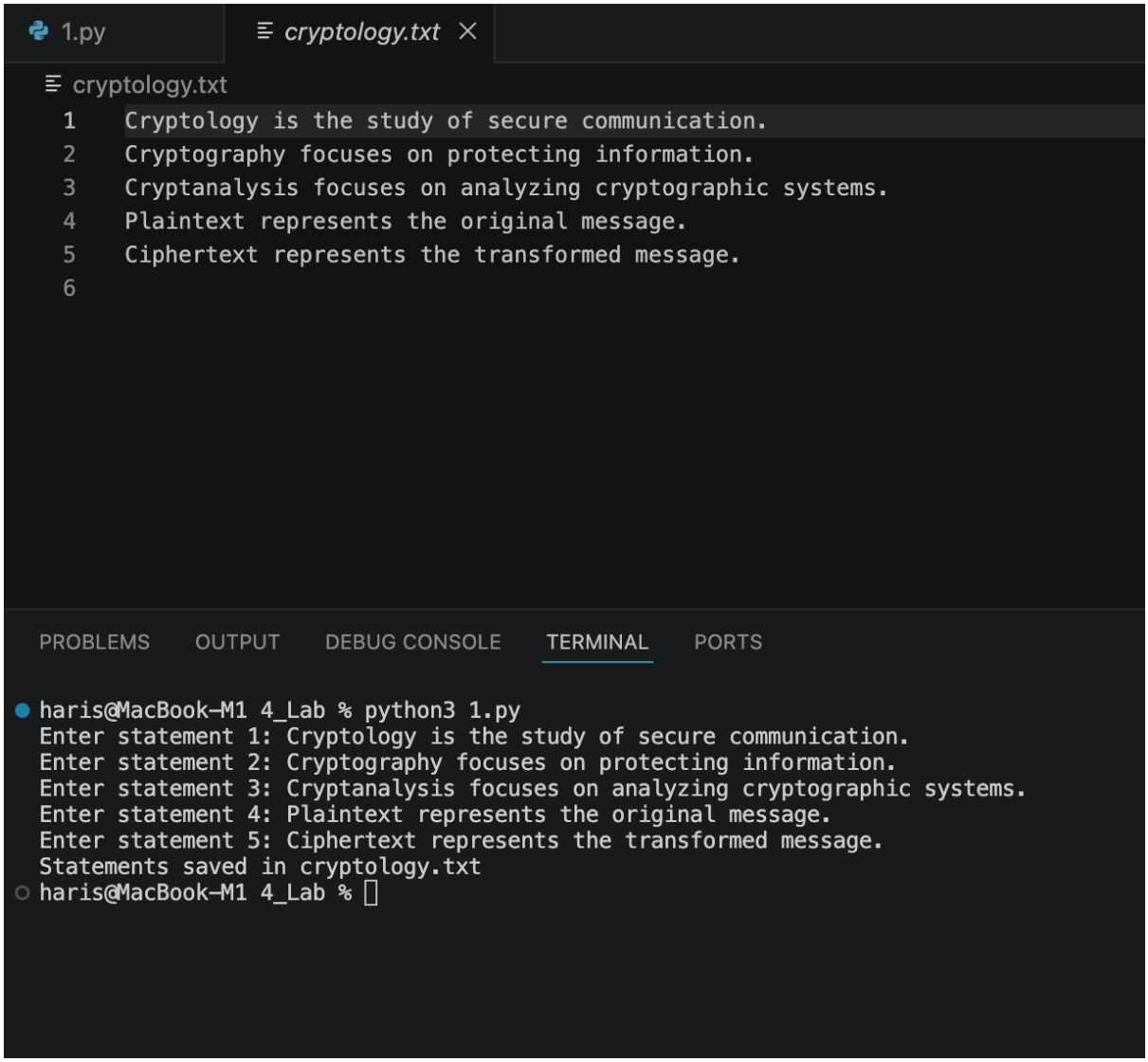
```
# Accept five statements from the user and store them in cryptology.txt

file = open("cryptology.txt", "w")

for i in range(5):
    statement = input("Enter statement " + str(i + 1) + ": ")
    file.write(statement + "\n")

file.close()

print("Statements saved in cryptology.txt")
```



2. Write a Python program to read cryptology.txt and display its complete contents.

```
# Read and display the complete contents of cryptology.txt

file = open("cryptology.txt", "r")
content = file.read()
print(content)
file.close()
```

```
haris@MacBook-M1 4_Lab % python3 2.py
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.

haris@MacBook-M1 4_Lab %
```

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

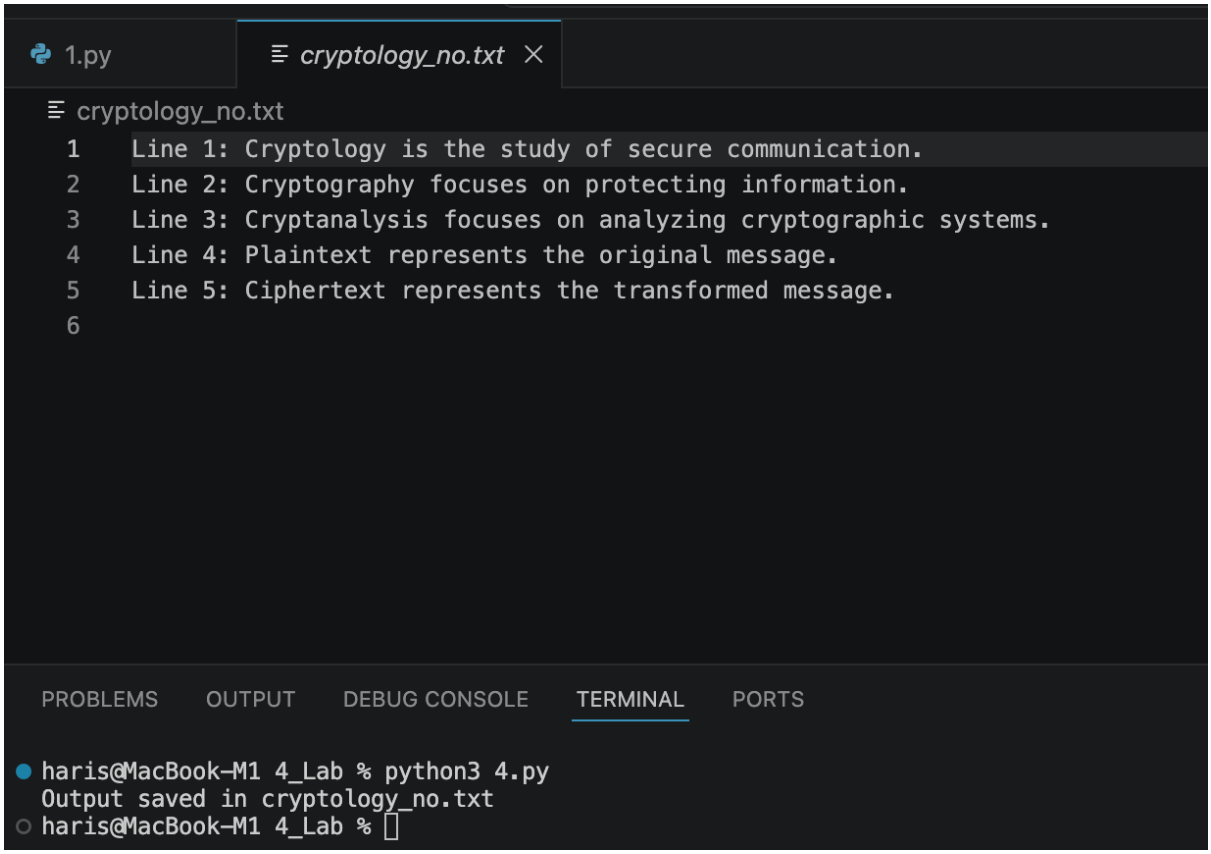
```
with open("cryptology.txt", "r") as file:
    for line_no, line in enumerate(file, start=1):
        print("Line", line_no, ":", line.strip())
```

```
haris@MacBook-M1 4_Lab % python3 3.py
Line 1 : Cryptology is the study of secure communication.
Line 2 : Cryptography focuses on protecting information.
Line 3 : Cryptanalysis focuses on analyzing cryptographic systems.
Line 4 : Plaintext represents the original message.
Line 5 : Ciphertext represents the transformed message.
haris@MacBook-M1 4_Lab %
```

4. write a program to store the output in a file called cryptology_no.txt

```
with open("cryptology.txt", "r") as infile, open("cryptology_no.txt", "w") as
outfile:
    for line_no, line in enumerate(infile, start=1):
        outfile.write(f"Line {line_no}: {line.strip()}\n")

print("Output saved in cryptology_no.txt")
```



5. Write a program to read cryptology.txt and determine:

Number of lines
Number of words
Number of characters

```
with open("cryptology.txt", "r") as file:
    content = file.readlines()

line_count = len(content)
word_count = 0
char_count = 0

for line in content:
    word_count += len(line.split())
    char_count += len(line)

print("Number of lines:", line_count)
print("Number of words:", word_count)
print("Number of characters:", char_count)
```

```
haris@MacBook-M1 4_Lab % python3 5.py
Number of lines: 5
Number of words: 28
Number of characters: 245
haris@MacBook-M1 4_Lab %
```

6. Read cryptology_no.txt and count the number of:

Uppercase letters
Lowercase letters
Digits
Spaces
Special characters

```
with open("cryptology_no.txt", "r") as file:
    text = file.read()

upper = lower = digit = space = special = 0

for ch in text:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digit += 1
    elif ch.isspace():
        space += 1
    else:
        special += 1

print("Uppercase letters:", upper)
print("Lowercase letters:", lower)
print("Digits:", digit)
print("Spaces:", space)
print("Special characters:", special)
```

```
haris@MacBook-M1 4_Lab % python3 6.py
Uppercase letters: 10
Lowercase letters: 222
Digits: 5
Spaces: 38
Special characters: 10
haris@MacBook-M1 4_Lab %
```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```
word = input("Enter a word to search: ")
```

```
with open("cryptology.txt", "r") as file:
    text = file.read()

if word in text:
    print("Word found in cryptology.txt")
else:
    print("Word not found in cryptology.txt")
```

```
haris@MacBook-M1 4_Lab % python3 7.py
Enter a word to search: focuses
Word found in cryptology.txt
haris@MacBook-M1 4_Lab %
```

8. Read cryptology.txt and display only the lines containing the word message.

```
word = "message"

with open("cryptology.txt", "r") as file:
    for line in file:
        if word in line:
            print(line, end="")
```

```
haris@MacBook-M1 4_Lab % python3 8.py
Plaintext represents the original message.
Ciphertext represents the transformed message.
haris@MacBook-M1 4_Lab %
```

9. Write a Python program to read cryptology.txt, convert all alphabetic characters to uppercase, and store the result in a new file called normalized.txt.

```
with open("cryptology.txt", "r") as f1, open("normalized.txt", "w") as f2:
    for line in f1:
        f2.write(line.upper())

print("Converted text saved to normalized.txt")
```

normalized.txt

```
1   CRYPTOLOGY IS THE STUDY OF SECURE COMMUNICATION.
2   CRYPTOGRAPHY FOCUSES ON PROTECTING INFORMATION.
3   CRYPTANALYSIS FOCUSES ON ANALYZING CRYPTOGRAPHIC SYSTEMS.
4   PLAINTEXT REPRESENTS THE ORIGINAL MESSAGE.
5   CIPHERTEXT REPRESENTS THE TRANSFORMED MESSAGE.
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
haris@MacBook-M1 4_Lab % python3 9.py
Converted text saved to normalized.txt
haris@MacBook-M1 4_Lab %
```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology_no.txt file.

```
with open("cryptology_no.txt", "r") as file:
    for line in file:
        line = line.strip()
        if line.isalpha():
            print(line)
```

11. Write a program to get the string from the user and reverse the string.

```
text = input("Enter a string: ")
print(text[::-1])
```

```
haris@MacBook-M1 4_Lab % python3 11.py
Enter a string: something
gnihtemos
haris@MacBook-M1 4_Lab %
```

12. Improve the above code to reverse the string from the file.

```
with open("cryptology.txt", "r") as file:
    text = file.read()
```

```
print(text[::-1])
```

```
haris@MacBook-M1 4_Lab % python3 12.py

  .egassem demrofsnart eht stneserper txetrehpiC
  .egassem lanigiro eht stneserper txetnialP
  .smetsys cihpargotpyrc gnizylana no sesucof sisylanatpyrC
  .noitamrofni gnitcetorp no sesucof yhpargotpyrC
  .noitacinummoc eruces fo yduts eht si ygolotpyrC
haris@MacBook-M1 4_Lab %
```

13. Simulate the tac command in Python.
or
Create a Python program that accepts five lines of text related to cryptology from the user and stores them in input.txt. Read the file and determine the number of lines, words, alphabetic characters, digits, spaces, and special characters. Convert the text to uppercase and store the processed contents in processed.txt. Finally, read and display the contents of processed.txt.

```
# Accept 5 lines and store in input.txt
with open("input.txt", "w") as f:
    for i in range(5):
        line = input("Enter line {}: ".format(i + 1))
        f.write(line + "\n")

# Read file and count
with open("input.txt", "r") as f:
    lines = f.readlines()

num_lines = len(lines)
num_words = 0
alpha = digits = spaces = special = 0

for line in lines:
    num_words += len(line.split())
    for ch in line:
        if ch.isalpha():
            alpha += 1
        elif ch.isdigit():
            digits += 1
        elif ch.isspace():
            spaces += 1
        else:
            special += 1

print("Lines:", num_lines)
print("Words:", num_words)
print("Alphabetic characters:", alpha)
print("Digits:", digits)
```

```python
print("Spaces:", spaces)
print("Special characters:", special)

# Convert to uppercase and store in processed.txt
with open("input.txt", "r") as f1, open("processed.txt", "w") as f2:
    for line in f1:
        f2.write(line.upper())

# Display processed.txt
with open("processed.txt", "r") as f:
    print("\nContents of processed.txt:")
    print(f.read())
```

processed.txt

```
1   HELLO WORLD
2   PYTHON 3 IS FUN
3   EMAIL ME AT TEST@EXAMPLE.COM
4   LINE WITH #SPECIAL$ CHARACTERS!
5   12345 ABC
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
haris@MacBook-M1 4_Lab % python3 13.py
Enter line 1: Hello World
Enter line 2: Python 3 is fun
Enter line 3: Email me at test@example.com
Enter line 4: Line with #special$ characters!
Enter line 5: 12345 abc
Lines: 5
Words: 16
Alphabetic characters: 72
Digits: 6
Spaces: 16
Special characters: 5

Contents of processed.txt:
HELLO WORLD
PYTHON 3 IS FUN
EMAIL ME AT TEST@EXAMPLE.COM
LINE WITH #SPECIAL$ CHARACTERS!
12345 ABC

haris@MacBook-M1 4_Lab %
```

Level 2: Regex

security_data.txt:

User alice logged in from 192.168.10.15
User bob logged in from 10.10.5.21
Contact admin@securitylab.com
Failed login attempts: 5
Session ID: SEC-2026-1045
Hash: a94f3c2d8e71b05f
Port 443 connection established
User admin failed authentication

1. Write a Python program to read security_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```python
import re

with open("security_data.txt", "r") as file:
    data = file.read()

pattern = r"\b(?:\d{1,3}\.){3}\d{1,3}\b"
ips = re.findall(pattern, data)

for ip in ips:
    print(ip)
```

```
haris@MacBook-M1 4_Lab % python3 14.py
192.168.10.15
10.10.5.21
haris@MacBook-M1 4_Lab %
```

2. Write a Python program to identify and display all email-address-like patterns present in security_data.txt.

```python
import re

with open("security_data.txt", "r") as file:
    data = file.read()

pattern = r"\b[\w.-]+@[\w.-]+\.\w+\b"
emails = re.findall(pattern, data)

for email in emails:
    print(email)
```

```
of directory
haris@MacBook-M1 4_Lab % python3 15.py
admin@securitylab.com
haris@MacBook-M1 4_Lab %
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

```python
import re

with open("security_data.txt", "r") as file:
    data = file.read()

pattern = r"\bSEC-\d{4}-\d{4}\b"
session_ids = re.findall(pattern, data)

for sid in session_ids:
    print(sid)
```

```
haris@MacBook-M1 4_Lab % python3 16.py
SEC-2026-1045
haris@MacBook-M1 4_Lab %
```

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```python
import re

with open("security_data.txt", "r") as file:
    text = file.read()

hex_values = re.findall(r"\b[0-9a-fA-F]+\b", text)

for value in hex_values:
    print(value)
```

```
haris@MacBook-M1 4_Lab % python3 17.py
192
168
10
15
10
10
5
21
5
2026
1045
a94f3c2d8e71b05f
443
haris@MacBook-M1 4_Lab %
```

5. Write a Python program to read security_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
import re

with open("security_data.txt", "r") as file:
    for line in file:
        if re.search(r"failed", line, re.IGNORECASE):
            print(line, end="")
```

```
haris@MacBook-M1 4_Lab % python3 18.py
Failed login attempts: 5
User admin failed authentication
haris@MacBook-M1 4_Lab %
```

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```
import re

with open("security_data.txt", "r") as file:
    text = file.read()

# Port number
port_match = re.search(r"Port (\d+)", text)
if port_match:
    print(port_match.group(1))
```

e

```
haris@MacBook-M1 4_Lab % python3 19.py
443
haris@MacBook-M1 4_Lab %
```